day 2

Day 2 — Filtering & Sorting + Data Types & Schemas

Part 1: DBMS Theory (15 min)

Tables, Rows, Columns You've already seen this in action. Formally: a **table** (relation) is a collection of related data. Each **row** (tuple) is one record. Each **column** (attribute) has a name and a **data type** that constrains what values it can hold.

Core Data Types in PostgreSQL:

Type	What it stores	Example
`INTEGER`	Whole numbers	`42` , `-7`
`SERIAL`	Auto-incrementing integer (for IDs)	`1, 2, 3...`
`VARCHAR(n)`	Text up to n characters	`'hello'`
`TEXT`	Unlimited text	Long descriptions
`BOOLEAN`	`TRUE` / `FALSE` / `NULL`	`TRUE`
`DATE`	Date only	`'2024-03-15'`
`TIMESTAMP`	Date + time	`'2024-03-15 14:30:00'`
`NUMERIC(p,s)`	Exact decimal (p digits, s after decimal)	`99.95`

What is a Schema? A schema is a namespace — a container that groups related tables. PostgreSQL puts everything in a schema called `public` by default. When you write `SELECT * FROM employees` , it's actually `SELECT * FROM public.employees` . Schemas help organize large databases (e.g., `sales.orders` vs `hr.employees`).

NULL — the most misunderstood concept in SQL `NULL` doesn't mean zero or empty string. It means **unknown / missing / not applicable**. A salary of `NULL` means "we don't know their salary" — not "they earn 0." This distinction matters because NULL behaves differently from every other value, as you'll see today.

Part 2: New Filtering Tools (30 min)

2.1 — IN (matches any value in a list)

Instead of chaining OR conditions:

```
-- The long way
WHERE city = 'Mumbai' OR city = 'Delhi' OR city = 'Bangalore'

-- The clean way
WHERE city IN ('Mumbai', 'Delhi', 'Bangalore')
```

```
SELECT name, city
FROM employees
WHERE city IN ('Mumbai', 'Delhi');
```

`NOT IN` does the inverse:

```sql
SELECT name, city
FROM employees
WHERE city NOT IN ('Mumbai', 'Delhi');
```

2.2 — BETWEEN (range inclusive)

```sql
-- Salary between 70000 and 90000 (includes both endpoints)
SELECT name, salary
FROM employees
WHERE salary BETWEEN 70000 AND 90000;
```

This is equivalent to `salary >= 70000 AND salary <= 90000`.

Works with dates too:

```sql
SELECT name, hire_date
FROM employees
WHERE hire_date BETWEEN '2021-01-01' AND '2022-12-31';
```

2.3 — LIKE (pattern matching)

`%` matches any sequence of characters. `_` matches exactly one character.

```sql
-- Names starting with 'A'
SELECT name FROM employees
WHERE name LIKE 'A%';
```

Returns: Arun Kumar, Anita Desai, Amit Joshi

```sql
-- Names containing 'Kumar'
SELECT name FROM employees
WHERE name LIKE '%Kumar%';
```

```sql
-- Names where second character is 'r'
SELECT name FROM employees
WHERE name LIKE '_r%';
```

Returns: Arun Kumar, Priya Sharma

LIKE is case-sensitive in PostgreSQL. Use `ILIKE` for case-insensitive matching:

```sql
WHERE name ILIKE '%kumar%'   -- matches 'Kumar', 'KUMAR', 'kumar'
```

`ILIKE` is PostgreSQL-specific — won't work in MySQL or others.

2.4 — IS NULL / IS NOT NULL

This is a trap that catches everyone. You **cannot** use `=` to check for NULL.

```sql
-- WRONG — this returns nothing, even if NULLs exist
WHERE salary = NULL

-- CORRECT
WHERE salary IS NULL
```

```
-- Also correct
WHERE salary IS NOT NULL
```

Why? Because `NULL = NULL` evaluates to `NULL` (unknown), not `TRUE`. Any comparison with NULL gives NULL, which is treated as false in a WHERE clause.

Let's add a row with a NULL to see this:

```
INSERT INTO employees (name, department, salary, city, hire_date)
VALUES ('Ravi Iyer', 'Sales', NULL, 'Bangalore', '2024-01-15');
```

```
-- Find employees with missing salary
SELECT name, salary
FROM employees
WHERE salary IS NULL;
```

Returns: Ravi Iyer

2.5 — ORDER BY (sorting)

```
-- Sort by salary, lowest first (ascending is the default)
SELECT name, salary
FROM employees
WHERE salary IS NOT NULL
ORDER BY salary;

-- Highest first
ORDER BY salary DESC;

-- Sort by department alphabetically, then by salary highest first within each
ORDER BY department, salary DESC;
```

NULLs in sorting: PostgreSQL puts NULLs last in ascending order, first in descending. You can control this:

```
ORDER BY salary ASC NULLS FIRST
ORDER BY salary DESC NULLS LAST
```

2.6 — ORDER BY + LIMIT = "Top N"

```
-- Top 3 highest paid employees
SELECT name, salary
FROM employees
WHERE salary IS NOT NULL
ORDER BY salary DESC
LIMIT 3;
```

Result:

```
    name      | salary
-------------+--------
 Karan Mehta | 102000
 Arun Kumar  |  95000
 Neha Gupta  |  91000
```

Mental model — the full order of execution so far:

```
FROM       →  pick the table
WHERE      →  filter rows
ORDER BY   →  sort the surviving rows
LIMIT      →  chop to N rows
SELECT     →  pick which columns to show
```

Part 3: Exercises (45 min)

Q1. Show the name, department, and salary of employees in Engineering or HR, sorted by salary from highest to lowest.

Q2. Find all employees hired between 2020-01-01 and 2021-12-31, showing name and hire_date, sorted by hire_date earliest first.

Q3. Find all employees whose name contains "Patel" or "Sharma" (use a single query — try both LIKE and IN approaches, and note which one works).

Q4. Show the name and salary of the 2 lowest-paid employees who have a known salary (i.e., exclude NULLs).

Q5. Find all employees NOT in Engineering whose name starts with a vowel (A, E, I, O, U). Show name and department. (Hint: you might need OR chains or multiple LIKEs — there's no clean shortcut yet.)

Post your queries when done, or say **"show answers"**.